



CCP6224 OPERATING SYSTEM

ASSIGNMENT (DESIGN REPORT)

GROUP 5 (T16L)

NAME	STUDENT ID
ALESHA NOOR BINTI MOHAMED SALIM	242UC244SU
ARIANA FALISYA BINTI AIDY @ MOHAMED SHAWAL	242UC244PW
AMIRAH BALQIS BINTI AZLAN	242UC244LC
WAFIYYAH DIANNAH BINTI HUSSIN	242UC244T9

Table of Contents

1 Game Description	1
3 Hybrid Architecture	3
4 IPC Mechanism	4
5 Synchronization Strategy	5
6 Logger Design	6
7 RR Scheduler	7
8 Persistence Strategy	8
9 Multi-Game Handling	9
10 Testing Evidence	10
11 Screenshots	11

1 Game Description

Yahtzee, considered one of the most popular and recognised dice games in the world, especially in the West, is a turn-based dice game for one or more players.

In the game, players use five six-sided dice to score points across 13 categories. Players aim for the highest total score by rolling dice up to three times per turn, utilizing strategy to fill in different categories, with the ultimate goal of rolling five-of-a-kind— also known as a “Yahtzee”.

The game ends once all 13 categories have been filled by each player.

1.1 Upper Section Combinations and Scoring

- **Ones:** Sum of all dice showing 1.
- **Twos:** Sum of all dice showing 2.
- **Threes:** Sum of all dice showing 3.
- **Fours:** Sum of all dice showing 4.
- **Fives:** Sum of all dice showing 5.
- **Sixes:** Sum of all dice showing 6.

At the end of the game, the sum of all the above combinations will be calculated. **If it is 63 or more, the player will get a bonus of 35 points.**

1.2 Lower Section Combinations and Scoring

- **Three-of-a-Kind:** At least three dice show the same number. Score the sum of all five dice.
- **Four-of-a-Kind:** At least four dice show the same number. Score the sum of all five dice.
- **Full House:** Three dice of one number and two dice of another (e.g., 2-2-2-6-6). Scores a fixed 25 points.
- **Small Straight:** Four sequential dice (e.g., 1-2-3-4, 2-3-4-5, or 3-4-5-6). Scores a fixed 30 points.
- **Large Straight:** Five sequential dice (e.g., 1-2-3-4-5 or 2-3-4-5-6). Scores a fixed 40 points.
- **Yahtzee:** Five-of-a-kind. Scores a fixed 50 points.
- **Chance:** Any combination. Score the sum of all five dice; usually used when a roll doesn't fit other categories.

1.3 Multiple Yahtzees

There are additional, special rules whenever a player happens to roll a second “Yahtzee”.

1. **When the player has already scored a 50 in the “Yahtzee” box:** They’ll not only gain a 100-bonus points in the Yahtzee box, but they’ll also get a “Joker”. This means that the player can choose another move for the Yahtzee they’ve just gotten.
 - Situation 1: **If the number the player got Yahtzees with has not been filled out in the upper section, then they must choose that.** For example, if the player gets an additional Yahtzee with 2s, and they haven't filled out their 2s in the Upper Section, then they must choose that and get 10 points for it.
 - Situation 2: **If the upper section box is already filled, then the player can choose any of the lower region boxes, and they will be scored as normal.** A “Yahtzee” is a superset of 3 of a kind, 4 of a kind, full house and chance; but they can also choose to score small or large straight and will get the normal 30 and 40 points for those.
2. **When the player has already scored a 0 in the “Yahtzee” box:** The player will not get a 100-point bonus, but they do get a “Joker”. The player can choose a move following the rules described above for Jokers.

1.4 End of the Game

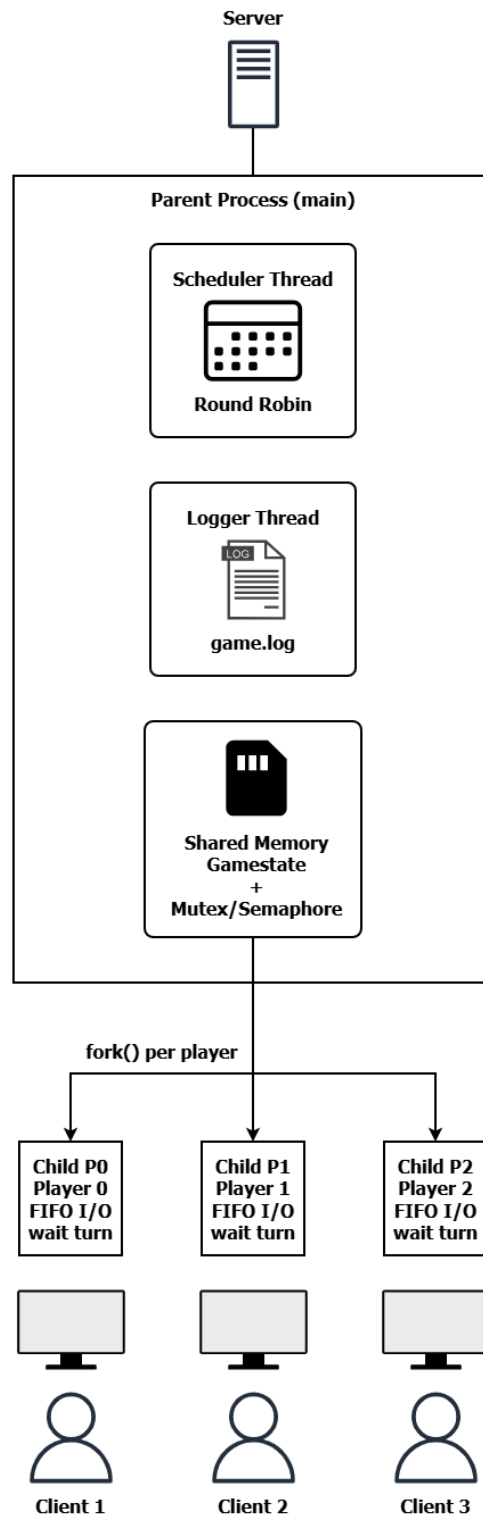
At the end of the game, the scores from all 13 categories and additional bonus points will be added altogether. **The winner will be determined by who has the highest score.**

2 Deployment Mode

In our reinterpretation of the game using **C-language**, we have decided to make it playable in “**single-machine mode**”. This means that all components will be residing on the same host, utilizing **Inter-Process Communication (IPC)** to exchange data between distinct “**Client**” and “**Server**” processes.

3 Hybrid Architecture

The diagram below shows the various processes involved with our game. It illustrates a client-server system architecture that uses a combination of multithreading, multi-processing, and various Inter-Process Communication (IPC) and synchronization mechanisms.



4 IPC Mechanism & Shared Memory Layout

Our multiplayer Yahtzee game uses a hybrid inter-process communication (IPC) model that combines named pipes (FIFOs) for message exchange and POSIX shared memory for maintaining uniform game state. This design ensures that communication remains simple while shared memory allows fast and synchronized access to gameplay data.

4.1 IPC Mechanism

When a client joins, it sends its FIFO information through the server's main FIFO at **/tmp/yahtzee/server_fifo**. The server then sets up two private FIFOs for that client, one for receiving player input and one for sending back server messages. After registration, the server calls **fork()** to create a dedicated child process to handle that specific client. This separates each client's communication flow, preventing interference between players while allowing multiple players to connect concurrently.

4.2 Shared Memory

When memory is created using **shm_open** and mapped into all server processes **mmap**. It stores the complete **GameState** structure containing all gameplay variables. Since all child processes inherit this shared memory, updates by a client are constantly visible to all others. This approach removes the need to repeatedly transmit full game data through pipes and makes synchronization significantly faster.

4.3 Shared Memory Layout

The core of the shared memory region is a large struct called **GameState**, which combines all essential data for gameplay.

Categories	Data
Session control	active_players, target_players, host_payer_id, game_started, game_round, game_finished
Participation & connectivity	participants[MAX_PLAYER], participants_count, player_connected[MAX_PLAYERS]
Turn management	current_turn, turn_deadline[MAX_PLAYERS], turn_active[MAX_PLAYERS]
Per-player scoring state	player_scores[player][category][...]
Persistence-like counters	total_wins[MAX_PLAYERS]
Synchronization primitives inside shared memory	pthread_mutex_t game_mutex, pthread_mutex_t log_mutex, sem_t turn_sem[MAX_PLAYERS], sem_t turn_done_sem [MAX_PLAYERS]

5 Synchronization Strategy

< how mutexes/semaphores coordinate threads and processes>

Since multiple server child processes run at the same time, the system needs a clear synchronization strategy to prevent data races and to maintain correct turn ordering. This is achieved through a combination of mutexes and semaphores with each handling a different aspect of process coordination.

5.1 Mutex for Protecting Shared Data

The main mutex, `game_mutex`

6 Logger Design

<thread structure, queue, safety>

6.1 Purpose of the Logger

The logger is responsible for keeping a record of all significant runtime events of the Yahtzee game for debugging, auditing, and testing purposes. These events include client connections and disconnections, turn changes, dice rolls, scoring actions, and game termination. The logging mechanism is implemented as a dedicated thread within the parent server process to ensure concurrent, ordered, and non-blocking logging.

6.2 Logger Architecture

The logging system follows a producer-consumer design:

Role	Description
Producer (Parent Server Process)	Generates log messages for server-level events such as server startup, turn scheduling, and game termination.
Producer (Forked Client Handler Processes)	Generate log messages for client-related events including player connections, dice rolls, scoring actions, and disconnections.
Consumer (Logger Thread)	Consumes log messages from the shared queue and writes them sequentially to game.log

This design helps prevent multiple processes from writing directly to the log file, which could otherwise result in race conditions and corrupted output.

6.3 Log Queue and Synchronization

Log messages are first placed into a shared in-memory log queue. Access to this queue is protected using a process-shared mutex, ensuring mutual exclusion across threads and forked processes. A semaphore or condition variable is used to notify the logger thread whenever new log messages are available. All synchronization primitives used by the logger are initialized with the **PTHREAD_PROCESS_SHARED** attribute so that they are visible and usable across process boundaries.

6.4 The Logging Workflow

1. A game event occurs (example: a player rolls dice or a turn changes).
2. The producing process formats a timestamped log message.
3. The message is pushed into the shared log queue while holding the mutex.
4. The logger thread is signaled and wakes up.
5. The logger thread removes messages from the queue and appends them sequentially to **game.log**.

6.5 Safety and Performance Guarantees

- Only the logger thread performs file I/O operations.
- Log entries are written sequentially, ensuring chronological order.
- Gameplay processes never block on disk writes.
- Log interleaving and file corruption are fully prevented.

This design satisfies the requirement for a concurrent, thread-safe, and non-blocking logger.

7 RR Scheduler

The server implements a Round-Robin (RR) scheduling strategy to coordinate turn execution among multiple player processes. A dedicated scheduler thread runs concurrently within the server process and is responsible for managing turn order, enforcing fairness, and handling timing constraints.

The scheduler maintains a cyclic order over all players participating in the current game. During each scheduling cycle, it selects the next eligible player based on the following conditions: **the player must be part of the current match, must still be connected, and must not have completed all scoring categories**. Once a suitable player is identified, the scheduler grants the turn by signalling a semaphore associated with that player.

Each turn is bounded by a fixed time quantum of 60 seconds. The scheduler monitors the turn and advances to the next player when the turn is completed, the time quantum expires, or the player disconnects. This ensures that no single player can monopolize execution time and that gameplay proceeds fairly and predictably.

The turn execution flow is as follows;

1. Scheduler selects the next eligible player
2. Scheduler signals the player using a semaphore
3. The player performs actions within the time quantum
4. Turn ends on completion, timeout, or disconnection
5. Scheduler advances to the next player

Synchronization between the scheduler thread and player processes is achieved using **POSIX semaphores stored in shared memory**. In the event of an unexpected player disconnection, the server immediately updates the shared state, forfeits the remaining actions of that player, and skips the turn without delay. This design prevents “ghost turns” and ensures uninterrupted gameplay for the remaining participants.

8 Persistence Strategy

<Describe the file format, the loading/saving mechanism for **scores.txt**, and the synchronization used to protect the in-memory score data>

8.1 Purpose of Persistence

The persistence strategy ensures that player scores are retained across multiple games and server restarts. Persistent storage allows the server to maintain historical scoring data beyond a single execution of the program.

8.2 Persistent Data and File Format

Player score data is stored in a text file named **scores.txt** . Each line in the file represents one player's record using the following format:

<player_name>,<score>,<timestamp>

This simple line-based format is human-readable and easy to parse using standard C file I/O functions.

8.3 Loading Mechanism

Upon server startup, the server checks for the existence of **scores.txt** .

- If the file exists, its contents are loaded into a shared in-memory score structure.
- If the file does not exist, a new file is created.

The in-memory score structure is stored in POSIX shared memory so that it is accessible by the parent server process, its threads, and all forked child processes.

8.4 Updating Scores

At the conclusion of each game, the server determines the winning player. The corresponding score entry in shared memory is then updated atomically. To prevent race conditions, all read and write operations on the score structure are protected using a **process-shared mutex**. This ensures correct behavior even if multiple processes attempt to access the score data simultaneously.

8.5 Saving Mechanism

Updated scores are written back to **scores.txt** either:

- After each completed game, and/or
- During server shutdown via signal handler (example: **SIGNIT**)

The server acquires the persistence mutex before writing to the file, writes all score entries, flushes the output, and then releases the mutex

8.6 Reliability and Multi-Game Support

- Only completed games trigger score updates.
- Partial or crashed games do not affect persistent data.
- The persistence mechanism supports multiple consecutive games without requiring a server restart.

This design ensures safe, synchronized, and reliable score persistence across sessions.

9 Multi-Game Handling

The system supports multiple consecutive games without restarting the server. When a game concludes, the server finalises scores, determines the winner, and resets all game-related state variables such as turn order, player participation flags, and completion status. Persistent data, including total win counts, is preserved across games.

Game Reset Behaviour:

Component	Action After Game Ends
Turn state	Reset
Player status	Reinitialized
Scheduler	Continues running
Total wins	Preserved
Server process	Remains active

The server process remains active between game sessions, allowing clients to reconnect or initiate a rematch without requiring server reinitialization. This approach improves system stability, reduces overhead, and enables seamless transitions between games.

10 Testing Evidence

<gameplay screenshots + sample game.log>

11 Screenshots

<screenshots of each client view, part of the logger, and persistent storage content>